

```

1  # Kobo Deluxe Sega CD32X - How It Boots and Runs
2
3  A guide for developers picking up the `kobo32x_hw` project (real hardware, Sega CD + 4MB
4  RAM cart). It explains, in order, what happens from power-on to the game running, which
5  processor does what, how the processors talk to each other, and the hardware rules that
6  were learned the hard way.
7
8  Sources: `subcpu/main.c`, `subcpu/hw_md.s`, `subcpu/crt0.s`, `subcpu/crt.s`,
9  `subcpu/cd.s`, `subcpu/scd.c`, `subcpu/Makefile`, `sh2/crt0.s`, `sh2/sh2_main.c`.
10
11 > **Marked "inferred":** a step that happens inside the toolchain's generic boot block (
12   `/opt/toolchains/sega/bootblocks/US_BOOT.BIN`). That file is not part of this project;
13   its behaviour is deduced from what our code expects.
14
15 ---
16
17 ## 1. The four processors and who runs what
18
19 | Processor | Program | Source | Role |
20 |---|---|---|---|
21 | **Sega CD Sub-CPU** (68000, 12.5 MHz) | `APP.BIN` | `subcpu/crt0.s`, `main.c`, `scd.c`,
22   `cd.s` | **The coordinator.** Reads the CD (BIOS), drives the CD audio and PCM chip,
23   writes the cart, and answers the SH2's requests. |
24 | **Genesis 68000** (7.6 MHz) | listener copied into its RAM at `0xFF1000` |
25   `subcpu/hw_md.s` (`md_init_start`...`md_init_end`) | **A command slave** for the Sub-CPU.
26   It does what only the Genesis side can do: Genesis VDP and text, controllers, the 32X
27   registers and frame buffer, and the cart window. |
28 | **32X master SH2** (23 MHz) | `KOB032X.BIN` (built as `D32XR.32x`) | `sh2/crt0.s`,
29   `sh2/sh2_main.c` | **The game:** logic, drawing, menus. Asks the Sub-CPU for everything
30   it cannot do itself. |
31 | **32X slave SH2** | same program | `sh2/crt0.s` (`sec_start`) | Started and parked in
32   an idle loop. Not used by the game. |
33
34 **Key idea.** The SH2 cannot reach the CD or the Sega CD hardware. The Sub-CPU cannot
35 reach the 32X or the Genesis VDP. The Genesis 68000 can reach both sides, but has no CD
36 access. So every request travels **SH2 → (32X COMM registers) → Sub-CPU → (Sega CD comm
37 registers) → Genesis 68000 → hardware**, and the replies come back the same way.
38
39 ---
40
41 ## 2. The disc
42
43 `subcpu/Makefile`, target `cd`, builds an ISO-9660 image with `genisoimage`:
44
45 - **System area / boot sector:** `US_BOOT.BIN` (`-generic-boot`), the toolchain's generic
46   Sega CD boot block (security code, IP for the Genesis, SP for the Sub-CPU).
47 - **Files on the disc:**
48   - `APP.BIN`: our Sub-CPU program (`CDROMPlayer.bin`, linked with `cd.ld`).
49   - `KOB032X.BIN`: the SH2 program. It is also embedded inside `APP.BIN` via `files.s`,
50     which is the copy that is actually booted.
51   - `KOB0GFX.BIN`: all graphics (201,540 bytes, one file).
52   - `KOBOSFX.BIN`: the PCM sound pack (64 KB image + table).
53   - `IMAGE.RAW`: the splash picture, 320×200 at 15 bits.
54 - **Audio tracks:** `make_mixed_cd2.py CDROMPlayer.iso Track02.wav Track03.wav
55   Track04.wav` turns the ISO into a mixed-mode BIN/CUE.
56   - Track 1: data
57   - Track 2: title / menu / game music
58   - Track 3: credits
59   - Track 4: level transition
60
61 Build everything with `kobo32x_hw/build.sh`. It refuses to build unless both programs are
62 in cart mode.
63
64 ---
65
66 ## 3. Boot, step by step

```

```

46
47 ### Stage 0 - Sega CD BIOS
48 The BIOS checks the disc's system area and loads the boot block's IP (Genesis side) and
   SP (Sub-CPU side).
49
50 ### Stage 1 - the generic boot block loads APP.BIN *(inferred)*
51 - The boot block's SP loads `APP.BIN` into Sub-CPU program RAM and calls its entry point
   `_start` (`crt0.s`: "CD startup at 0x8000").
52 - The boot block's IP keeps the Genesis 68000 waiting until there is code for it in Word
   RAM, then jumps there.
53
54 ### Stage 2 - Sub-CPU start-up (`subcpu/crt0.s`)
55
56 **`_start`:**
57 1. Interrupts off.
58 2. Clear BSS.
59 3. Save the BIOS stack pointer.
60 4. Set our own stack.
61 5. Call `init_hardware`.
62 6. Run C initialisers.
63 7. Call `main()`.
64 8. If `main()` ever returns, restore the BIOS stack and return.
65
66 **`init_hardware`** (in `hw_md.s`, runs on the Sub-CPU) copies the **Genesis code** (
   `md_init_start` ... `md_init_end`) to `0x080000`. That is Word RAM, which the Genesis sees
   at `0x200000`.
67
68 **`main()`**, first lines:
69 - `write_byte(0x8003, read_byte(0x8003) & ~1); while (read_byte(0x8003) & 2);`
70 - This hands Word RAM over to the Genesis side, so the Genesis can run the code from it.
71
72 ### Stage 3 - the Genesis 68000 becomes a listener (`hw_md.s`)
73 1. The Genesis jumps into Word RAM at `md_init_start`.
74 2. `md_init_start` copies `md_start`...`md_init_end` to **`0xFF1000`** (Genesis work RAM)
   and jumps there. Address rule: runtime address = `0xFF1000 + (file offset - 0x174)`.
75 3. `md_init_hw` sets up the Genesis VDP:
76 - display on, V-interrupt on (register 1 = `0x8174`);
77 - Chilly's font in VRAM;
78 - a three-line palette: line 0 white `0x0CCC`, line 1 green (now **black**, see §8),
   line 2 red;
79 - installs `vblank_int` in the BIOS jump table at `0xFFFFD08`.
80 4. **The listener loop** (`cmd_loop` / `md_check`) waits for commands from the Sub-CPU
   (§4.2).
81 5. **`vblank_int`, every frame:**
82 - reads pad 1 and pad 2 → `0xA12018` / `0xA1201A`;
83 - increments the tick counter → `0xA1201C`;
84 - writes the counter to 32X COMM6 once the 32X is active (diagnostic heartbeat);
85 - raises the Sub-CPU's level-2 interrupt (`0xA12000 |= 0x0100`).
86
87 ### Stage 4 - waking up the 32X (`hw_md.s` `init_32x`)
88 The Sub-CPU copies the SH2 program (`sh2_app_start`, from `files.s`) into Word RAM, gives
   the bank to the Genesis (`switch_banks`), and sends `MD_CMD_INIT_32X (0x200000, length)`
   .
89
90 The Genesis 68000 then:
91
92 1. checks for **`"MARS"`** at `0xA130EC` (is a 32X there? else returns -2);
93 2. waits for the adapter to be ready (bit 7 of `0xA15101`; timeout → -5);
94 3. sets **ADEN** (`0xA15101 = 1`) and waits a moment;
95 4. clears COMM0-COMM6, sets ADEN + reset release (`0xA15101 = 3`);
96 5. gives the frame buffer to the Genesis (clears FM, bit 7 of `0xA15100`);
97 6. clears the 32X control registers;
98 7. clears **both frame buffers** (toggling FS in `0xA1518B`) and the 32X palette (
   `0xA15200`);
99 8. checks COMM0 for **`"SDER"` (32X boot ROM self-test error → returns -3);
100 9. copies the SH2 program from Word RAM to the **frame buffer** at `0x840000`;

```

```

101 10. gives the frame buffer to the SH2s (sets FM);
102 11. writes **`"_CD_"`** to COMM0 - the signal for the SH2 boot ROMs to boot from the
    frame buffer;
103 12. waits for **`"M_OK"`** in COMM0 (master) and **`"S_OK"`** in COMM4 (slave);
104 13. clears COMM0/COMM4 and returns.
105
106 Chilly's **white** start-up lines ("Activated 32X", "Cleared 32X", "32X Waiting", "CD
    BOOT TABLE", ...) are printed by `init_32x` (`message0`...`message6`).
107
108 ### Stage 5 - the SH2 boot ROMs and `sh2/crt0.s`
109 **The 32X boot ROMs** (described in `sh2/crt0.s`):
110 - **Master:** waits for `"_CD_"`, reads the header at the start of the program in the
    frame buffer (destination `0x06000000`, length, entry points, VBRs), copies the program
    to **SDRAM `0x06000000`**, writes `"M_OK"` to COMM0, and jumps to `pri_start`.
111 - **Slave:** waits for `"_CD_"` and the master's `M_OK`, writes `"S_OK"` to COMM4, and
    jumps to `sec_start`.
112
113 **`pri_start`:**
114 1. sets up the stack;
115 2. clears `.bss`;
116 3. writes **COMM4 = 0** (releases the slave);
117 4. turns the cache on;
118 5. calls `main()`.
119
120 It also writes diagnostic checkpoints to COMM10 (`0x0001`...`0x0003`).
121
122 ### Stage 6 - the start handshake (COMM10)
123 **SH2 `main()`:**
124 - writes checkpoints `0x0004`...`0x0007` to COMM10;
125 - enables interrupts;
126 - writes **`0x0AAA`** to COMM10 ("SH2 ready");
127 - waits for **`0x0BBB`**.
128
129 **Sub-CPU `main()``, after INIT_32X:**
130 - writes `0x1234` to COMM4 (legacy, nobody waits for it any more - see §7);
131 - polls COMM10 (through the Genesis) for `0x0AAA` (3 s timeout);
132 - checks the cart (`safe_cart_present`);
133 - writes **`0x0BBB`**.
134
135 The SH2 then takes the frame buffer (`take_fb`) and starts loading. The `0x0AAA`/`0x0BBB`
    exchange works in either order: each value stays in COMM10 until the other side reads
    it. (The old design also waited for COMM4 = `0x1234`; that raced with `crt0.s` clearing
    COMM4 and was removed in K28.)
136
137 ### Stage 7 - the Sub-CPU's main loop and the game's start-up
138 The Sub-CPU then:
139 - runs `InitCD`;
140 - prints the CD track info;
141 - enters its **main loop**, serving SH2 commands (§4.1).
142
143 The SH2 start-up posts a status code in COMM12 after each step. The Sub-CPU shows it as
    `STATE:xxxx` when the green text is on.
144
145 | Code | Step |
146 |---|---|
147 | `4B01` | Load start: `CMD 38` opens `KOBGFX.BIN`; `CMD 46` copies it in 32 KB chunks
    into the **cart** at 2 MB (cart mode, Chilly's RV method, every chunk verified). |
148 | `4B02` | Pack in the cart. |
149 | `4B03` | Graphics copied from the cart into SDRAM (the SH2 reads the cart at
    `0x22000000`). |
150 | `4B04` | Sound pack loaded into the PCM chip (`CMD 53`). |
151 | `4B05` | High scores and options read from backup RAM (`CMD 56`/`57`). |
152 | `4B51`-`4B54` | Palette loaded, level 1 built, 256-colour mode and text set (`CMD 52`),
    buffer swaps done. |
153 | `4B6n` / `4B7n` | Tile set *n* copied from the cart / written into both frame buffers
    (spare memory). |

```

```

154 | `4B06`, `4B07` | Tile sets done; panel drawn. |
155 | `4B08` | **Music started** (`CMD 45`). Rule: **no CD data reads after this point.** |
156 | `4B10` | Main loop running: menu, game. |
157
158 If `KOBO_SPLASH 1`, the splash (`CMD 51`, `IMAGE.RAW` straight into the frame buffer) is
shown before the loading, and the Genesis text layer is switched off at that point.
159
160 ---
161
162 ## 4. How the processors talk at runtime
163
164 ### 4.1 SH2 ↔ Sub-CPU: the 32X COMM registers
165
166 | Register | SH2 address | Genesis address | Use |
167 |---|---|---|---|
168 | COMM0 | `0x20004020` | `0xA15120` | **Command** from the SH2 (low byte), plus a
sound-effect id in the high byte for `CMD 48`. Cleared to 0 by the Sub-CPU when done. |
169 | COMM2 | `0x20004022` | `0xA15122` | Command parameter (file number, track, chunk, word
index...). |
170 | COMM4 | `0x20004024` | `0xA15124` | Start-up only (`S_OK`, legacy `0x1234`). |
171 | COMM6 | `0x20004026` | `0xA15126` | Genesis vblank counter (diagnostic heartbeat `V`).
|
172 | COMM8/10 | `0x20004028`/`2A` | `0xA15128`/`2A` | Replies: pad (COMM8) + Genesis tick
counter (COMM10) for `CMD 48`; file length; start-up handshake in COMM10. |
173 | COMM12 | `0x2000402C` | `0xA1512C` | SH2 status code (`STATE`); 16-bit command replies
(cart chunk result, sound/backup-RAM results). |
174 | COMM14 | `0x2000402E` | `0xA1512E` | Sub-CPU heartbeat (`P`): drive-status nibble +
pass counter. |
175
176 **SH2 commands** (`sh2_main.c` sends, `main.c` handles):
177
178 | Cmd | Name | Action |
179 |---|---|---|
180 | 38 | open file | COMM2 = file number. Reply: length in COMM8. |
181 | 45 | play CD track | COMM2 = track; BIOS play-repeat. |
182 | 46 | chunk → cart | Cart mode; verified write. |
183 | 48 | pad | Fast path at the top of the Sub loop. Reply: pad + ticks in COMM8/10; the
high byte of COMM0 starts a sound effect. |
184 | 50 | file chunk → frame buffer | No-cart mode: 16 KB, off-screen. |
185 | 51 | file chunk → frame buffer | Splash: 32 KB, at an offset. |
186 | 52 | Genesis display on/off | Text layer. |
187 | 53 | load sound pack | → PCM chip. |
188 | 54 | stop music | BIOS `MSCSTOP`. |
189 | 56 / 57 / 58 / 59 | backup RAM | Load / get word / put word / save. |
190
191 **How a command works:**
192 1. The SH2 writes COMM2 (and COMM8) if needed, then the command into COMM0.
193 2. It waits (with ~20 µs pauses between reads) until COMM0 is 0.
194 3. It reads the reply.
195
196 ### 4.2 Sub-CPU ↔ Genesis 68000: the Sega CD comm registers (`hw_md.s`)
197 **Sub-CPU side (`do_md_cmdN` → `send_md_cmd`):**
198 1. writes up to four long arguments to `0xFF8020...0xFF802F`;
199 2. waits until the Genesis flag (`0xFF800E`) is 0;
200 3. writes the **command number** to its flag (`0xFF800F`);
201 4. waits for the Genesis to **echo** it in `0xFF800E`;
202 5. clears its flag;
203 6. waits for the Genesis flag to return to 0;
204 7. reads the result (`0xFF8010`).
205
206 **Genesis side (`md_check` → `dispatch` → `cmd_done`):**
207 1. reads the Sub flag (`0xA1200F`) **twice** and only acts if both reads match (HW10, see
S6);
208 2. echoes the command to `0xA1200E`;
209 3. pushes the four arguments (`0xA12020...`);
210 4. calls `cmd_table[cmd]` (`jsr`, returns at `0xFF10AE`);

```

```

211 5. writes the result (`0xA12010`);
212 6. waits (reading twice) for the Sub flag to be 0;
213 7. clears its own flag.
214
215 Commands used at runtime: `READ_32X_16/32` (19/21), `WRITE_32X_16/32` (20/22),
`SET_VDP_REG` (34), `PUT_STR` (9), `CHILLY_RV_MEMCPY` (33), `INIT_32X` (18),
`INSTALL_EXC` (31). Command 2 (`set_sr`) is disabled.
216
217 ---
218
219 ## 5. Memory maps
220
221 **SH2 SDRAM (256 KB at `0x06000000`)**
222
223 | Range | Contents |
224 |---|---|
225 | `0x06000000` | Program (must end below `0x0600C000`; checked at start-up, status `4BE6`
226 | `0x0600C000` | **WORK2:** fighter, boomerangs, bigship, rock2/3, enemy table (128), gun
list (512). |
227 | `0x0601EC00` | RLE explosions (explo5, bolt, bullet). |
228 | `0x06020000` | **WORK:** map, maze sites, tiles, stars (32), pack header, player, bolt,
logo (in the unused damage table), enemy4, beam, explosion, font, enemy1/3, rock1, ring,
enemy6, bomb, detonation, RLE explo3/4. |
229 | `0x0603CE00` | RLE rock and ring explosions. |
230 | up to `0x0603F400` | Master stack. |
231
232 **32X frame buffer (each of the two buffers, 128 KB)**
233
234 | Range | Contents |
235 |---|---|
236 | `0x00000`-`0x001FF` | Line table. |
237 | `0x00200`-`0x11A00` | The 320x224 picture. |
238 | `0x12000`-`0x16000` | No-cart loading window. |
239 | `0x16000`-`0x1D800` | Cache of the 5 tile sets. |
240 | `0x1D800`-`0x1FFFF` | Free. |
241
242 Write the frame buffer with **16-bit writes** ($6).
243
244 **Cart (4 MB):** `KOBGFX.BIN` at cart offset 2 MB (slot 16 × 128 KB). The Genesis
reaches it through the 32X bank window `0x900000` (bank register `0xA15104`); the SH2
reads it at `0x22000000 + offset`.
245
246 **Word RAM:** 1M mode. The Sub-CPU's bank is at `0x0C0000` and is used as a CD read
buffer and for text strings. `switch_banks()` swaps the banks between the CPUs.
247
248 ---
249
250 ## 6. Hardware rules learned on the console (Fusion shows none of these)
251
252 1. **Torn reads of the command flag.** The Genesis read `0xA1200F` while the Sub-CPU was
changing it and got 2 instead of 19. That ran `set_sr` with a garbage argument: `SR =
A71E`, trace bit on, interrupts off. Results: a red `68K EXC V09` (frame `A71E 00FF 10AE
...`), or a silent deadlock about 19 s after boot, because the echo never matched. The same
bug caused the old V70 crash (`00FF10AE`). **Fix:** read twice and compare, in
`md_check` and in `cmd_done`; disable unused dangerous commands.
253 2. **COMM registers:** do not hammer a register while the other CPU writes it. Use one
writer per register, poll with pauses, and avoid 32-bit writes that also hit another
CPU's half (COMM12/14).
254 3. **No CD data reads after the music starts.** The drive cannot read data and play audio
at the same time: the track keeps counting but is silent.
255 4. **Frame buffer:** write with 16-bit writes. Byte writes of `0x00` did not land in
Fusion.
256 5. **Colour `0x0000` is see-through** on the 32X (the Genesis layer shows). Draw black as
`0x0400`.
257 6. **The cart copy path:** keep the pack small; unused banks are written empty by

```

```

258 `kobo_gfx.py`.
259 7. **Diagnostic text is expensive:** each line swaps Word RAM and goes through the
Genesis. Reprinting every frame dropped the game to 1-2 fps.
259 8. **The SH2 build has no C library:** no variable shifts, no signed shifts by more than
1, no divisions by variables. The project's own `memset`/`memcpy`/`memmove` live in
`sh2_main.c`.
260 9. **The program must end below `0x0600C000`**, and big arrays must not go in `.bss`.
`crt0.s` clears `.bss` before releasing the slave; a long clear once raced the start
handshake.

261 ---
262
263
264 ## 7. Leftovers in the code (harmless, kept as history)
265
266 - **`main.c`, in the Sub-CPU's start-up sequence:**
267   - `InitCD()`, `memcpy(temp, Sub_Start, ...)` , `send_md_cmd0('I')`;
268   - "INITIALIZING SUB-CPU Sfx...", "68K LISTENER ACTIVE", "BOOTING SUB-CPU...";
269   - "(TEXT BLINKS TO SHOW SH2 IS ALIVE)".
270
271   These come from an older design where the program ran on the Genesis and booted the
   Sub-CPU itself. `scd.c` keeps that version under `#if 0`.
272 - **The `0x1234` write to COMM4:** nothing waits for it since K28.
273 - **"TOO TALL" after the file size:** a label from an old picture viewer. Not an error.
274 - **The `M` command in the listener:** jumps into Word RAM (`0x200000`); unused.
275 - **The `d7` branch in `cmd_loop`:** lets the Genesis serve SH2 commands directly; `d7`
   is 0, so it is unused.
276 - **`crt.s`:** a complete Sub-CPU system program (`SPMain` with its own command loop),
   embedded via `cd.s`. Its level-2 interrupt (`SPInt2`) is only `rts`.
277
278 ---
279
280 ## 8. Switches (`sh2/sh2_main.c` / `subcpu/main.c`)
281
282 | Switch | Release value | Meaning |
283 |---|---|---|
284 | `KOBO_NOCART` (both files) | 0 | Cart mode (hardware). 1 = no-cart (Fusion). |
285 | `KOBO_SPLASH` | 1 | Splash while loading. |
286 | `KOBO_HW_PCM` / `KOBO_HW_BRAM` | 1 / 1 | Sound effects / backup RAM. |
287 | `KOBO_HW_DIAG` | 0 | Bottom-left readout: P (Sub heartbeat + drive status), V (Genesis
   vblanks), C (command), T (pad timeouts), F (fps). |
288 | `KOBO_DIAG_TEXT` (`main.c`) | 0 | Green status text. The green colour is also set to
   **black** in `hw_md.s` `md_init_hw`; restore `0x00A0` to see it. |
289 | `KOBO_BOOT_TEXT` (`main.c`) | 1 | Keep the Genesis display on during boot (white
   lines). |
290 | `KOBO_HW_QUIET` (both) | 0 | Diagnostic: no CPU traffic after the menu appears. |
291 | `KOBO_FIXED_STEP` / `KOBO_STEP_MS` | 0 / 30 | Fixed-time game logic (off = one step per
   frame). |
292 | `KOBO_DEBUG_KEYS` | 0 | Debug extras. |
293

```